

Building AI solutions using Microsoft Foundry

Toma× Ka×trun, MVP

About

- Data scientist | Data Plaftorm MVP
- SQL Server, SAS, R, Python, C#, SAP, IBM, Databricks
- 20+years experience MSSQL, DEV, BI, DM, DS
- Frequent community speaker, book author
- Avid coffee drinker & bicycle junkie
- From Ljubljana, Slovenia – city with dragon brigde
- Work at the SLO bank and startup Putsimply.AI





http://tomaztsql.wordpress.com

tomaz.kastrun@gmail.com

@tomaz_tsql

/in/tomaztsql

http://github.com/tomaztk

https://mvp.microsoft.com/PublicProfile/50

Session material:

github.com/tomaztk/Azure-AI-services

Agenda



1 **Platform** × what Foundry is, resource model, portal

2 **Models** × catalog, deployment types, router, Responses API, fine-tuning

3 **Agents** × agent types, hosted agents, toolboxes, MCP, memory, multi-agent

4 **Knowledge and tools** × Foundry IQ, agentic retrieval, Foundry Tools

5 **Developer experience** × SDKs, VS Code, azd, GitHub, Agent Framework, Foundry Local

6 **Operate** × observability, evaluation, governance, reference architecture

7 **Demo** × invoice intake and data agent: architecture, data flow, Azure SQL, front end

8 **Real world questions** × from unstructured to structured and information validation

Demo: Invoice intake & Data agnet

Scan invoices with your phone, then ask the data



1) Document OCR with Azure AI Foundry
2) Containerized chatbot with access to Azure SQL database

1. Plaftorm

One Azure resource for models, agents and tools

aaa



What is Microsoft Foundry?



Microsoft Foundry unifies **agents, models and tools** under one Azure resource, with one set of access control, networking and policies. Tracing, monitoring, evaluations and guardrails are built in.

Azure AI Studio → **Azure AI Foundry** → **Microsoft Foundry**

2023 2024 2025

Same rename for the services: Cognitive Services → Azure AI Services → **Foundry Tools**

The capability map



Models
10,000+ models, model router, fine-tuning

Agents
Prompt, voice and hosted agents; workflows

Knowledge
Foundry IQ knowledge bases, memory

Tools
Toolboxes, MCP, 1,400+ catalog tools, Foundry Tools

Observability
Tracing, evaluations, AI red teaming, dashboards

Governance
Control Plane, guardrails, Entra ID, Purview, Defender

Developer surfaces
Portal · SDKs (Python, C#, JS/TS, Java) · azd · VS Code · GitHub Copilot and MCP

Sources: https://learn.microsoft.com/azure/foundry/concepts/capabilities

Foundry resource and projects

New Foundry



Foundry resource (Microsoft.CognitiveServices/accounts, kind AIServices)
Model deployments · networking and private link · policies · connections · quota · customer-managed keys

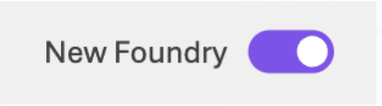
Project #1 invoice-agent
Agents · evaluations · files · traces · own access control

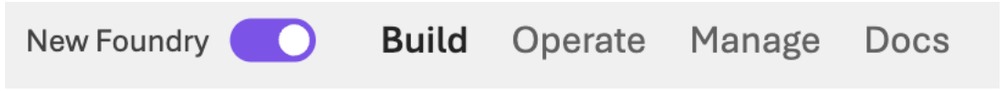
Project #2 support-bot
Agents · evaluations · files · traces · own access control

- Admins set security and governance once; developers create projects themselves
- Roles: Foundry User, Foundry Project Manager, Foundry Account Owner, Foundry Owner
- One project endpoint: https://<resource>.services.ai.azure.com/api/projects/<project>

The new Foundry portal (vs. the old)







Section	What you find there
Discover	Model catalog, benchmarks, leaderboards
Build	Agents, models and playgrounds, tools, knowledge, memory, guardrails, data, evaluations, fine-tuning
Operate	Fleet health, tracing, compliance, assets – across all projects
Manage	Quota, AI gateway, resource and project details, users, connected resources

Classic portal keeps hub-based projects, Azure OpenAI resources and prompt flow (retiring 20 Apr 2027).

2. Models

Catalog, deployment, routing and customization

Foundry Models (score-card) :)



10,000+

models in the catalog

~50

new models each month

35

models in the router pool

- **Sold by Azure**: OpenAI, Anthropic Claude, xAI Grok, DeepSeek, Meta, Mistral, Microsoft MAI and more
- **Partners and community**: thousands of models; provider sets licence and price
- **Instant access** (preview): call supported models by name, no deployment needed
- Filter by collection, region, deployment type, lifecycle, features (reasoning, tool calling), task
- Compare models side by side, check benchmarks and the model card before you pick

Deployment types



Type	Data processing	Billing	Best for
Global Standard	Any Azure region	Pay per token	Default; newest models, highest quota
Global Provisioned	Any Azure region	Reserved PTU	Predictable high throughput
Global Batch	Any Azure region	~50% off, 24 h	Large asynchronous jobs
Data Zone Standard / Provisioned / Batch	EU, US or APAC zone	Token / PTU / batch	Data-zone compliance
Standard	Azure geography	Pay per token	Geography compliance, low volume
Regional Provisioned	Azure geography	Reserved PTU	Geography + throughput
Developer	Any region, no SLA	Pay per token	Testing fine-tuned models (24 h)

Sources: https://learn.microsoft.com/azure/foundry/foundry-models/concepts/deployment-types

Model router and instant access



- **Model router**: one deployment that picks a model per request
  - 35 models in the pool (Oct 2026): OpenAI, Anthropic, xAI, open models via Fireworks
  - Automatic failover when a model endpoint is unstable
  - Routing mode and optional model subset; 32 regions
  - Session affinity and per-request routing metadata (preview)
  - Anthropic models must be deployed to your resource first
- **Instant access** (preview): call supported models by name without creating a deployment
- Good for prototypes; move to an explicit deployment type for production quotas and data residency

Responses API and the v1 endpoint



- One API for models and agents: the **Responses API**
- Standard **openai** SDK; no AzureOpenAI client, no api-version
- Conversations store items: messages, tool calls, outputs
- Synchronous by default: no run polling
- Keyless with Entra ID; API keys still work on /openai/v1
- Anthropic models can also use the Anthropic SDK

```
from openai import OpenAI
from azure.identity import (
    DefaultAzureCredential,
    get_bearer_token_provider)

token = get_bearer_token_provider(
    DefaultAzureCredential(),
    "https://cognitiveservices.azure.com/.default")

client = OpenAI(
    base_url=f"https://{RESOURCE}.services.ai.azure.com"
             "/openai/v1",
    api_key=token)

r = client.responses.create(
    model="gpt-5-mini",
    input="Summarize Foundry in one line")
print(r.output_text)
```

Fine-tuning



Method	Training signal	Example models
SFT – supervised	Prompts with desired responses	gpt-4.1, 4.1-mini, 4.1-nano, gpt-4o, gpt-4o-mini, Llama-3.3-70B, Qwen3-32B
DPO – preference	Preferred vs rejected responses	gpt-4.1 family, gpt-4o
RFT – reinforcement	A grader scores responses	o4-mini, gpt-5 (by invitation)

Training type	Where it runs
Standard	In the resource's region (data residency)
Data zone	Inside a data zone (US today)
Global	Any region, cheaper, faster queue
Developer	Idle capacity, cheapest, may be preempted

3. Agents

Foundry Agent Service

What is an AI Agent?



An agent uses a model to **decide and act**: it reasons about a goal, calls tools, checks the results and repeats until the task is done.

Input
User message · event · schedule · another agent

Agent

- **Model** — reasons and plans
- **Instructions** — goal, rules, limits
- **Tools** — act and fetch data
- **Knowledge** — your documents
- **Memory** — past turns, preferences

required: model, instructions, tools · optional: knowledge, memory

Output
Answer · action taken · handoff to a person

1 Reason → **2 Act (call a tool)** → **3 Observe the result** → **4 Repeat or finish**

Agent types



Type	What it is	Use when
Prompt agent	Model + instructions + tools, defined in portal or SDK	Instructions and built-in tools are enough
Voice prompt agent	Prompt agent that speaks and listens through Voice Live	You want real-time speech without hosting a pipeline
Hosted agent	Your code or container, run by Foundry in a per-session sandbox	Custom logic, any framework, stateful or long work
Workflow	Orchestration of several agents and steps	One task spans several agents (use Agent Framework)
Autopilot (preview)	Autonomous agent with its own Entra Agent ID in Teams and M365	Background work on events or schedules

Every agent is versioned: create_version() with an explicit kind.

A prompt agent in code



```
from azure.identity import DefaultAzureCredential
from azure.ai.projects import AIProjectClient
from azure.ai.projects.models import PromptAgentDefinition,
CodeInterpreterTool

project = AIProjectClient(endpoint=PROJECT_ENDPOINT,
credential=DefaultAzureCredential())

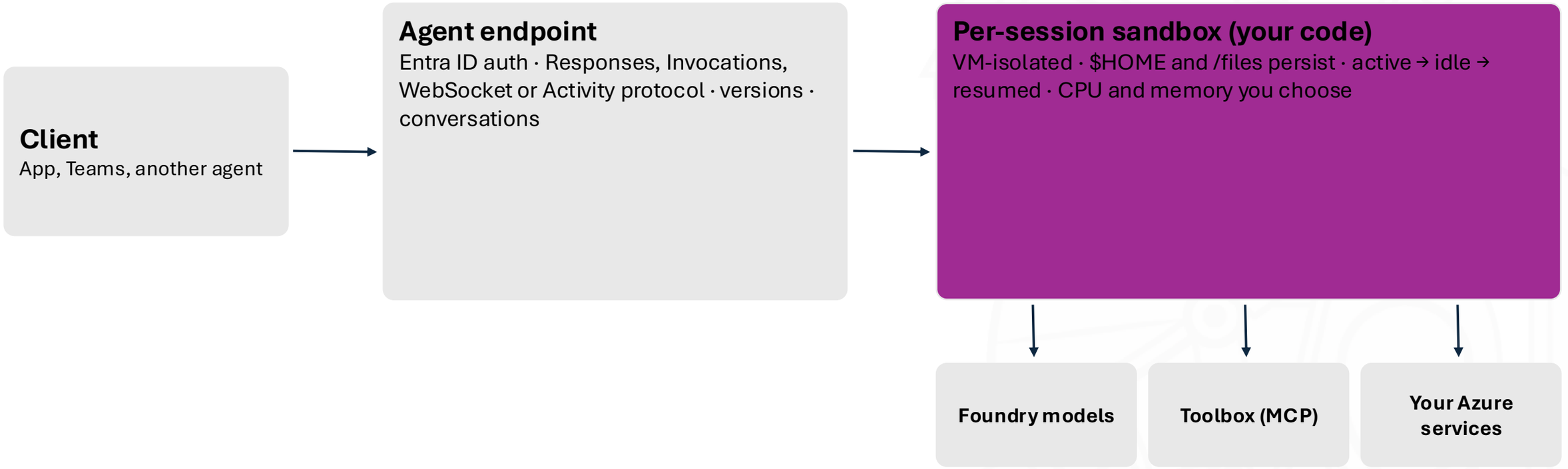
agent = project.agents.create_version(
    agent_name="math-agent",
    definition=PromptAgentDefinition(
        model="gpt-5-mini",
        instructions="Help with math. Use Code Interpreter for
charts.",
        tools=[CodeInterpreterTool()],
    ),
)

with project.get_openai_client(agent_name=agent.name) as openai:
    r = openai.responses.create(input="Plot y = 4x + 9")
    print(r.output_text)
```

- pip install "azure-ai-projects>=2.3.0"
- Versioned: each change is a new agent version
- Same agent works in the portal playground
- Swap CodeInterpreterTool for WebSearchTool, MCP or a toolbox

Hosted agents





- Any framework: Agent Framework, LangGraph, Semantic Kernel, GitHub Copilot SDK, Claude Agent SDK, custom
- Long-running and steerable agents (preview): durable state, reconnecting streams
- Private networking: bring your own VNet; private registry for new projects

Tools and toolboxes



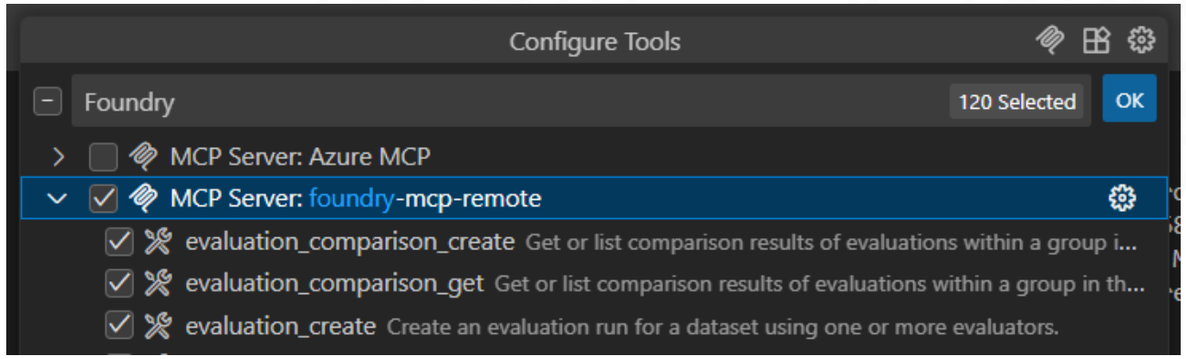
Tool	In a toolbox	Directly on an agent
MCP servers, OpenAPI, A2A	✓	✓
Web search, Azure AI Search, file search, code interpreter	✓	✓
Browser automation, Fabric IQ, Work IQ	✓	✓
Tool search, skills, reminders	✓	–
Function calling (runs in your app)	–	✓
Bing grounding, computer use, image generation, SharePoint, Azure Functions	–	✓

Tool catalog in the portal: 1,400+ tools, each labelled GA or preview.

MCP in Foundry



- **MCP tool** × connect any remote MCP server to an agent; choose which tools it may call
- **Toolbox** × one MCP endpoint for all of an agent's tools, usable from any MCP client
- **Foundry IQ** × knowledge bases are reachable over MCP
- **Foundry MCP Server** (preview) × https://mcp.ai.azure.com
  - 79 tools: agents, sessions, toolboxes, deployments, datasets, evaluations, quota
  - Use it from GitHub Copilot agent mode, Copilot CLI or Claude Code
  - Public endpoint only; for private networks build your own MCP server
- **Build your own**: e.g. Azure Functions MCP endpoint, Azure DevOps MCP Server



Memory, routines and publishing



- **Memory** (preview) × user, session and procedural memory across runs
- **Routines** (preview) × run any agent on a timer or schedule, e.g. nightly repo triage
- **Long-running agents** (preview) × crash-resilient work, steer an in-flight turn, human-in-the-loop approval
- **Agent optimizer** × improve prompt agents automatically, with cost and token reporting
- **Publishing** × Microsoft Teams and Microsoft 365 Copilot; expose agents over A2A
- **Agent identity** × Entra Agent ID; autopilots governed with Agent 365

Multi-agent systems



- **Microsoft Agent Framework workflows** × typed, graph-based orchestration in code; deploy as a hosted agent
- **A2A** × call other agents as tools, or expose a Foundry agent as an A2A endpoint (v1.0 GA)
- **Connected from Copilot Studio** × Copilot Studio agents can delegate to Foundry agents
- **Multi-agent orchestration on the Responses API** × for simpler patterns without a framework
- Declarative workflows in Foundry retire on **1 Dec 2026** × migrate to Agent Framework

4. Knowledge and tools

Foundry IQ and Foundry Tools

What is Foundry IQ?



- One knowledge base, many agents
- Automatic chunking, embeddings, scheduled refresh
- Queries run under the user's Entra identity

Agentic retrieval



- The LLM plans the query and splits it into subqueries
- Subqueries run in parallel across the chosen sources
- Results are semantically reranked and merged, with citations
- Medium effort can iterate when results are incomplete

Reasoning effort	LLM planning	Sources	Subqueries	Extras
Minimal	No	up to 10	–	Merge results only
Low	Yes	up to 3	up to 3	Answer synthesis (5k tokens)
Medium	Yes	up to 5	up to 5	Iterative search, synthesis (10k tokens)

Foundry Tools



Tool	What it does
Content Understanding	Documents, images, audio and video into structured fields with confidence scores
Speech and **Voice Live**	Speech to text, text to speech, translation; real-time voice agents
Language	PII detection, entities, health text; MCP server (preview)
Translator	100+ languages; API 2026-06-06 can use NMT or an LLM, with tone and gender controls
Document Intelligence and **Vision**	OCR, forms, invoices, receipts, image analysis
Content Safety	Harmful content, jailbreak and prompt-injection detection
Azure AI Search	Vector, hybrid and agentic retrieval; the engine behind Foundry IQ

5. Development and experience

SDKs, VS Code, azd, GitHub and local models

Pick your developer surface



Surface	Best for
Foundry portal	Explore models, prototype prompts, build prompt agents without code
SDKs	Applications in Python, C#, JavaScript/TypeScript, Java
Azure Developer CLI (azd ai)	Scaffold, run, test and deploy hosted agents from the terminal
Foundry Toolkit for VS Code	Build, debug, evaluate and deploy agents in the editor
Coding agents + MCP	GitHub Copilot or Claude Code with the Foundry Skill and MCP server
Foundry Canvas (preview)	Guided hosted-agent building in the GitHub Copilot App

```
winget install Microsoft.FoundryDevPack        # Windows
brew install --cask microsoft/foundry/devpack && foundry-devpack install   # macOS
```

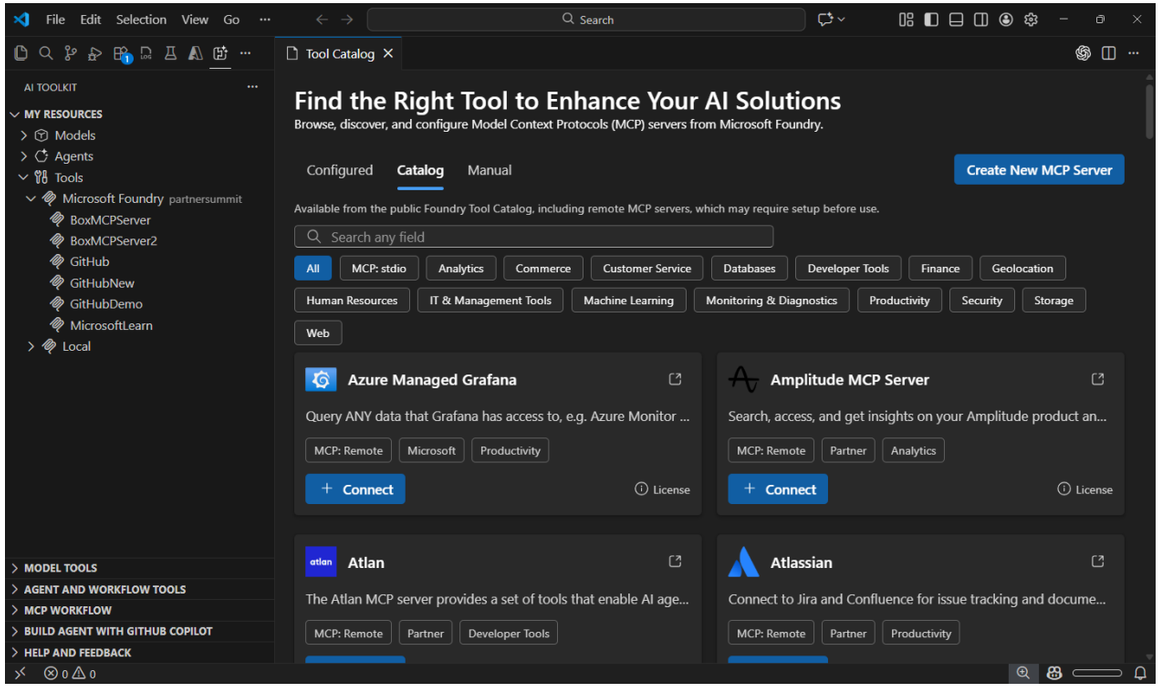
Foundry SDKs



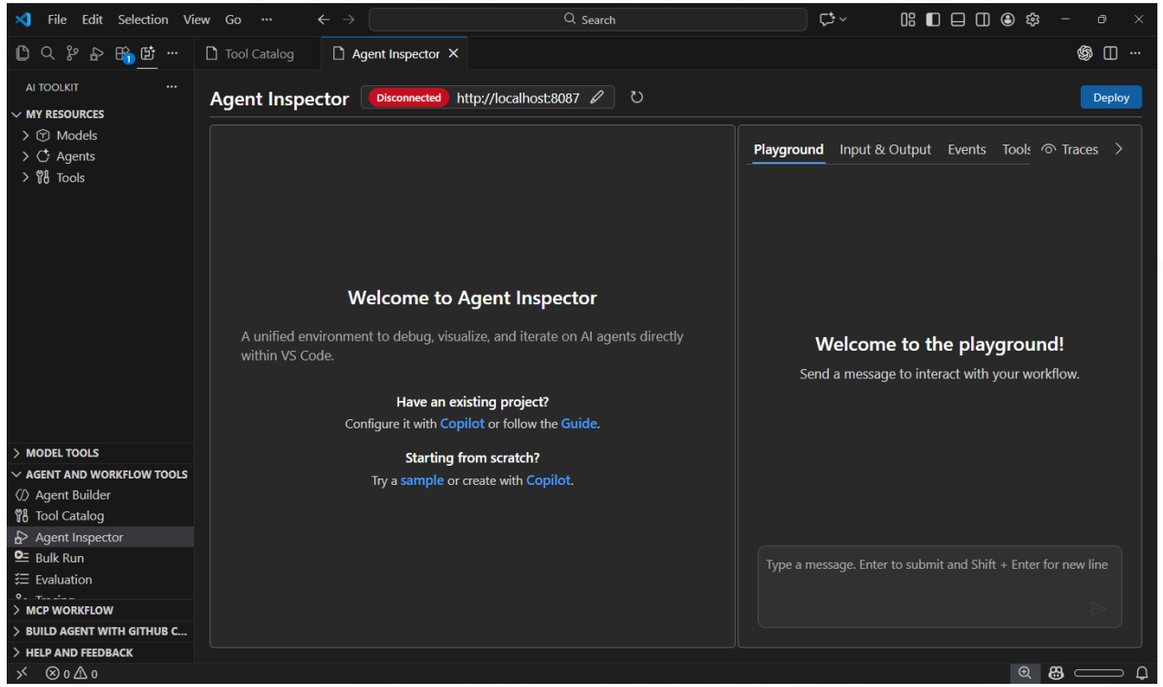
Language	Install	Runtime
Python	pip install "azure-ai-projects>=2.3.0" "openai>=3.0.0"	3.10+
C#	Azure.AI.Projects, Azure.AI.Projects.Agents, Azure.AI.Extensions.OpenAI	.NET 8+
JavaScript / TypeScript	npm install @azure/ai-projects @azure/identity	Node.js 22+
Java	com.azure:azure-ai-projects and azure-ai-agents 2.3.0	Java 17+

- One project client: agents, evaluations, datasets, indexes, connections, toolboxes
- get_openai_client() for the Responses API
- 2.x targets the new portal, 1.x the classic one – don't mix samples

Foundry Toolkit for VS Code



Tool Catalog: add MCP servers and Foundry tools to an agent



Agent Inspector: debug local agents, tool calls and traces

- Model catalog and playground, Agent Builder for prompt agents
- Hosted agents from samples or with Copilot; deploy to Foundry Agent Service
- Evaluations, traces and solution templates

azd for agents



azd ai agent init
scaffold from a template or your code

→

azd ai agent run
run locally with the inspector

→

azd ai agent deploy
provision and ship to Foundry

→

azd ai agent invoke
send test prompts

→

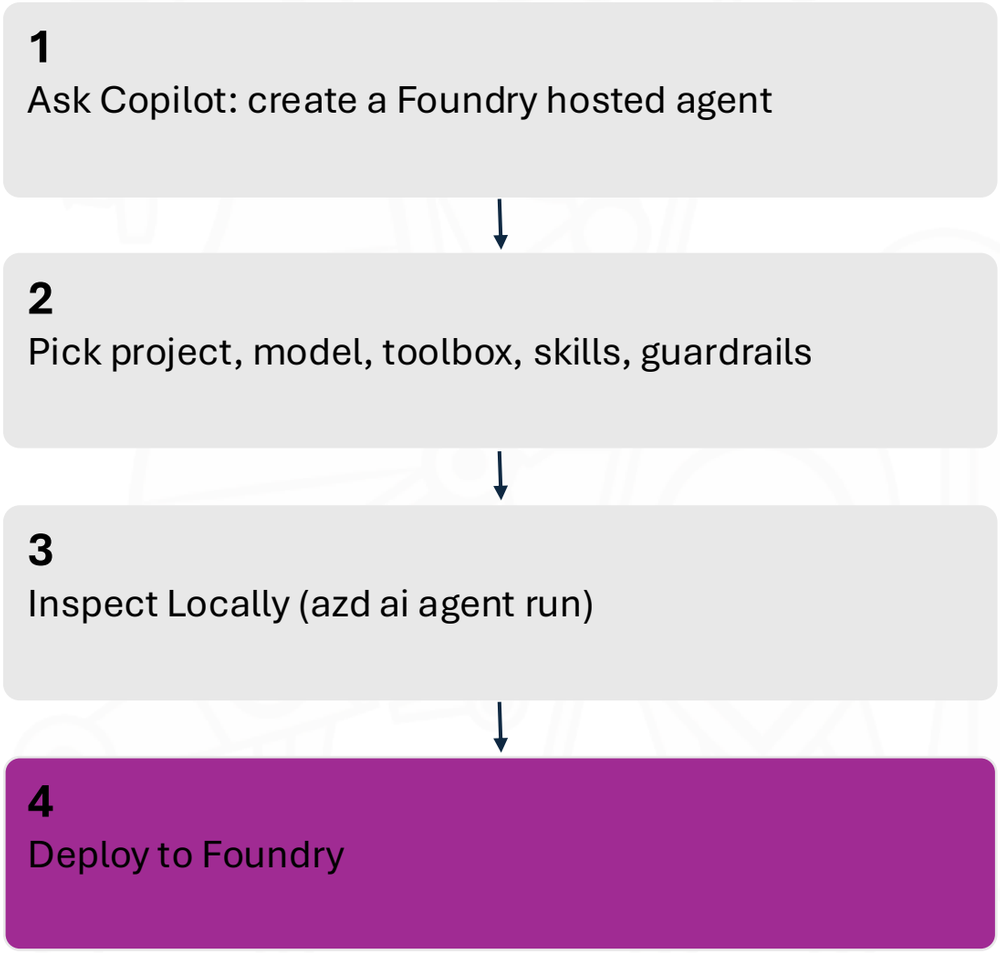
azd ai agent monitor
logs, versions, evals

Extension	Commands
azure.ai.agents	azd ai agent – ship agents from the terminal
azure.ai.toolboxes / skills	azd ai toolbox, azd ai skill – versioned tools and skills
azure.ai.routines	azd ai routine – timers, schedules, event triggers
azure.ai.connections / projects	azd ai connection, azd ai project – project context and connections
azure.ai.inspector	azd ai inspector – browser UI for local agents

GitHub and Foundry



- **GitHub Copilot SDK** (GA) – the Copilot runtime as an SDK; run Copilot SDK agents as hosted agents
- **Foundry Canvas** (preview) – design and deploy hosted agents from a side panel in the GitHub Copilot App
- **Microsoft Foundry Skill** – Copilot, Copilot CLI or Claude Code can build, deploy and evaluate agents
- **Foundry MCP Server** – manage Foundry from Copilot agent mode
- **CI/CD** – GitHub Actions with azd for hosted agents; evaluations as a quality gate



Foundry Canvas flow

Microsoft Agent Framework



- Open source, Python and .NET; successor to Semantic Kernel and AutoGen
- Agents with tools, sessions, middleware, telemetry; any model provider
- Graph-based workflows with checkpoints and human-in-the-loop
- Run anywhere, or deploy to Foundry as a hosted agent

```
pip install agent-framework-foundry azure-identity
pip install agent-framework --pre
```

Source: https://devblogs.microsoft.com/foundry/microsoft-agent-framework-reaches-release-candidate/

Foundry Local



- Run models **on the user's device**: Windows, macOS and Linux
- SDKs for C#, JavaScript, Python and Rust; runtime adds about 20 MB to your app
- Curated, quantized models: GPT OSS, Qwen, DeepSeek, Mistral, Phi, Whisper
- Picks the best hardware automatically: NPU, GPU (CUDA, WebGPU) or CPU
- OpenAI-compatible API, including the Responses format; optional local server
- Data never leaves the device; works offline; no per-token cost
- **Foundry Local on Azure Local** (preview): Kubernetes and Arc for on-premises inference

6. Stuff nobody talk about

Observability, governance and evaluations

Observability and evaluation



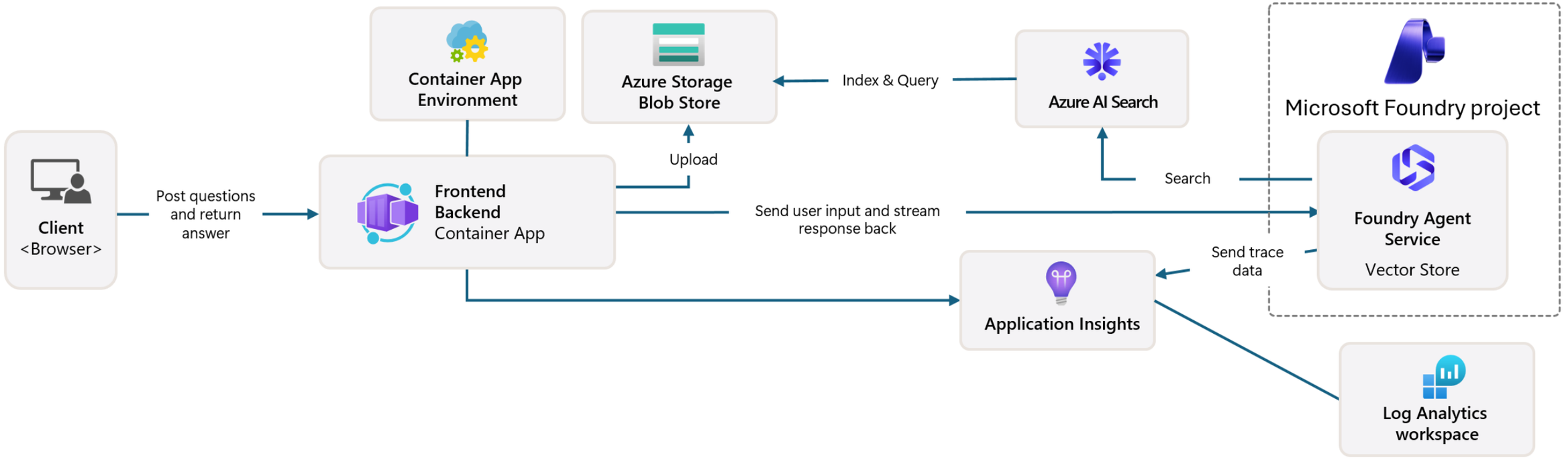
- **Tracing** with OpenTelemetry, for any framework; agent monitoring dashboard
- **Cloud evaluations** × quality, safety and agent evaluators: task adherence, intent resolution, tool call success, groundedness
- **Continuous evaluation** of live traffic; synthetic and trace-based datasets
- **AI red teaming agent** × automated adversarial scans; cluster analysis for root causes
- **Agent optimizer** × suggestions for prompts and model versions
- Playground evaluations are on by default and billed × switch them off if you don't need them

Governance and security



- **Foundry Control Plane** × one view of agents, models and tools across projects and platforms
  - Fleet KPIs, cost and token tracking, policies and bulk remediation
  - Defender and Purview alerts on the dashboard
- **Guardrails** × content filters, jailbreak and indirect prompt-injection protection
- **Identity** × Entra ID everywhere, agent identities, Foundry RBAC roles
- **Network** × private link, VNet for agents and toolboxes, customer-managed keys
- **Data** × choose Global, Data Zone or regional processing per deployment

Reference architecture: an agent web app



Source: Azure-Samples/get-started-with-ai-agents

- Frontend and backend in Container Apps; agent in a Foundry project
- Retrieval through Azure AI Search; traces to Application Insights
- One command to deploy: azd up

Demo: Invoice intake & Data agnet

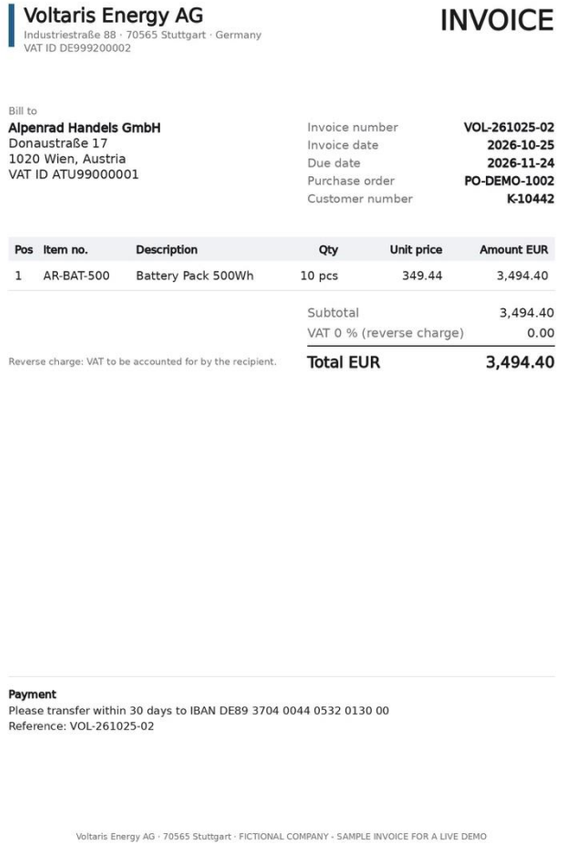
Scan invoices with your phone, then ask the data



1) Document OCR with Azure AI Foundry
2) Containerized chatbot with access to Azure SQL database

The demo in one slide

- **Scan**: photograph a printed invoice on the website
- **Read**: Content Understanding extracts the fields
- **Check**: rules compare it with PO, goods receipt and contract in Azure SQL
- **Decide**: booked automatically, or sent to a person with the reason
- **Ask**: a Foundry agent answers questions from the same database
- Fictional company: Alpenrad Handels GmbH, Vienna



Voltaris Energy AG
Industriestraße 88 · 70565 Stuttgart · Germany
VAT ID DE999200002

INVOICE

Bill to
Alpenrad Handels GmbH
Donaustraße 17
1020 Wien, Austria
VAT ID ATU99000001

Invoice number	VOL-261025-02
Invoice date	2026-10-25
Due date	2026-11-24
Purchase order	PO-DEMO-1002
Customer number	K-10442

Pos	Item no.	Description	Qty	Unit price	Amount EUR
1	AR-BAT-500	Battery Pack 500Wh	10 pcs	349.44	3,494.40

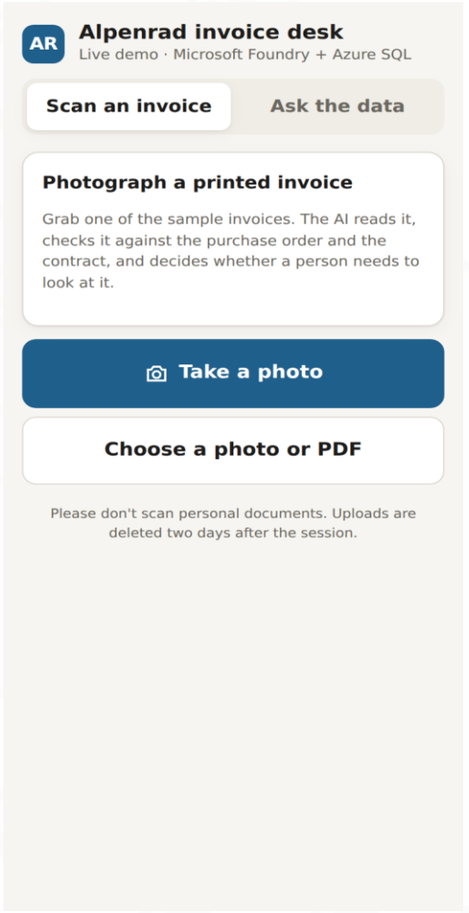
Subtotal	3,494.40
VAT 0 % (reverse charge)	0.00
Total EUR	**3,494.40**

Reverse charge: VAT to be accounted for by the recipient.

Payment
Please transfer within 30 days to IBAN DE89 3704 0044 0532 0130 00
Reference: VOL-261025-02

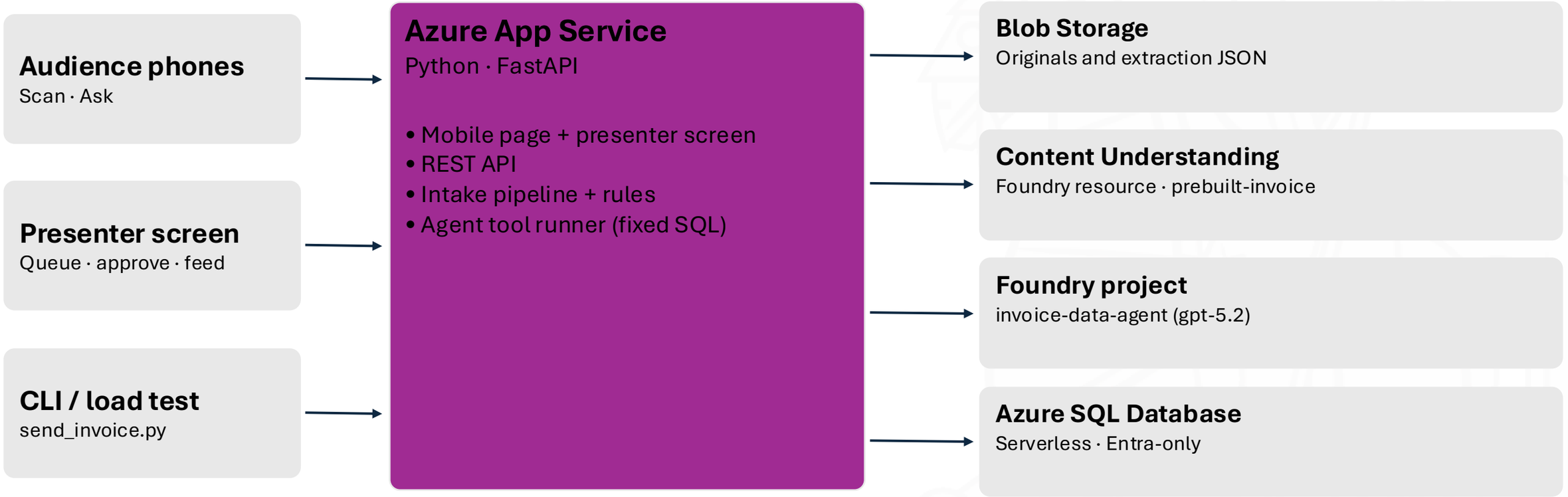
Voltaris Energy AG · 70565 Stuttgart · FICTIONAL COMPANY · SAMPLE INVOICE FOR A LIVE DEMO

Sample invoice 02 (price above contract)



Phone: scan page

Architecture



- Keyless everywhere: App Service managed identity + Entra ID; your own login when running locally
- Application Insights for logs and request timing · deployed with **azd up** (Bicep)

Intake data flow

1 Upload
POST /api/invoices · size, type and rate checks

→

2 Store
Original to Blob Storage

→

3 Extract
Content Understanding: fields + confidence

→

4 Check
Vendor, PO, receipt, contract from SQL; run rules

→

5 Decide
All pass → auto-approved; else to a person

→

6 Save
Header, lines, audit entry in one transaction

- The upload returns at once; processing runs as a background task
- The phone polls GET /api/invoices/{id} every 1.5 s: uploaded → reading → checking → done
- Typical time: 10–20 seconds per invoice
- At most 8 extractions at once, so 100 phones don't exhaust the quota
- Every failed check becomes a reason the phone and presenter screen show

Rules decide, not the model

- Is it an invoice? Known vendor?
- PO on the invoice, and does it exist?
- Lines match the PO? Goods received?
- Prices within contract tolerance?
- Totals add up? Duplicate?
- Read with enough confidence?
- Under the auto-approval limit (EUR 5,000)?

Sample	Result	Why
01 Brennerwerk	Auto-approved	Everything matches
02 Voltaris	Sent to a person	Unit price 12% above contract
03 Lago	Sent to a person	No purchase order number
04 Tauern	Sent to a person	Poor scan; 6 motors billed, 4 received
05 Nordkap	Auto-approved	Everything matches

The model reads; deterministic code decides. A mistake costs a review, not a wrong payment.

Talking to Azure SQL

Master data
Vendors · Products · Contracts · Customers · Regions · Calendar

Purchasing
PurchaseOrders · PurchaseOrderLines · GoodsReceipts

Invoices
Invoices · InvoiceLines · ApprovalLog

Sales
SalesOrders · SalesOrderLines

Audience
AudienceQuestions

Views for the agent
vw_InvoiceOverview · vw_InvoiceLineDeviations · vw_SalesLines

```
cur.execute("SELECT ... FROM dbo.vw_InvoiceOverview WHERE
Source = ? ...", (source,))
```

- **Write path** (intake): reference data in, then header + lines + audit entry in one transaction
- **Read path** (agent): five fixed, parameterized queries on the views
- **Driver**: mssql-python, Entra auth (managed identity in Azure, your login locally)
- **No passwords**: SQL server is Entra-only
- **Serverless**: pauses when idle; ~1 minute to wake

The data agent: question to answer

1 Phone: POST /api/ask

↓

2 App → Responses API (agent_reference)

↓

3 Agent returns a function call + arguments

↓

4 App runs fixed SQL on Azure SQL

↓

5 Rows back as function_call_output

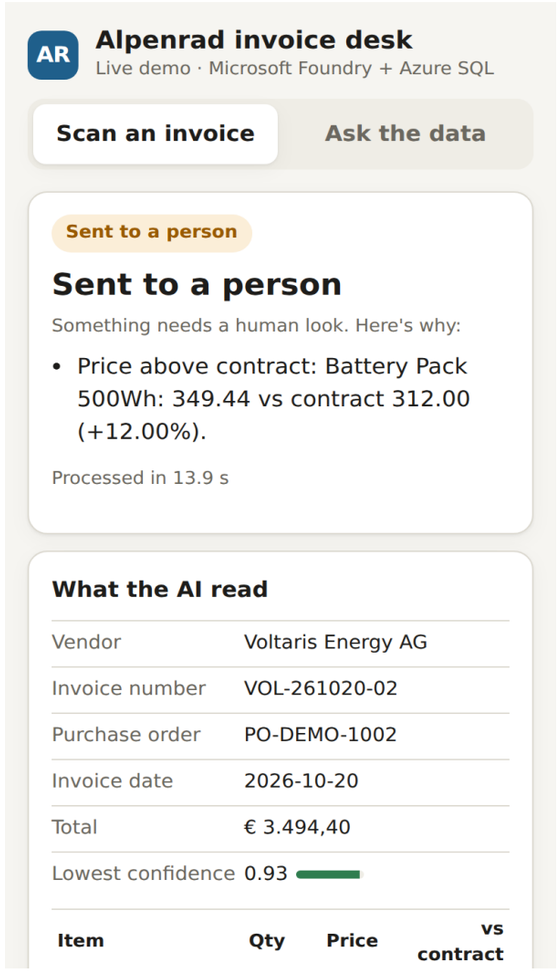
↓

6 Agent answers with numbers and a table

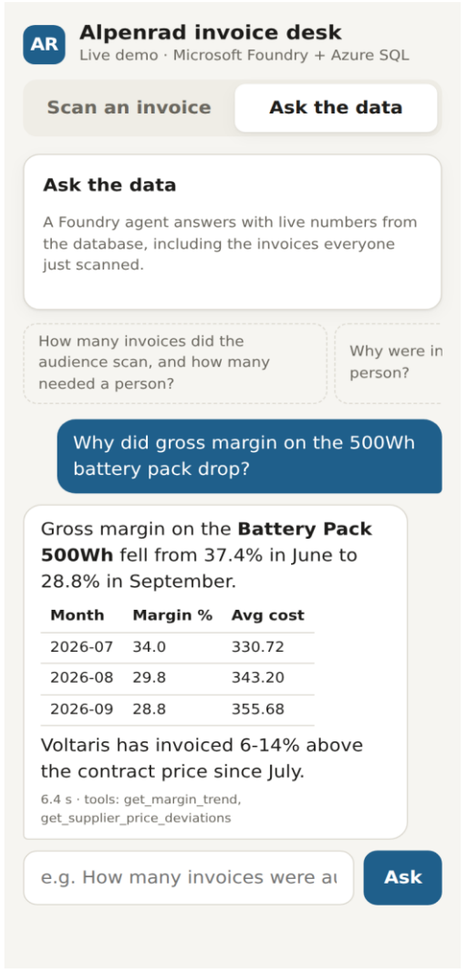
```
response = openai.responses.create(
    conversation=conv_id, input=question,
    extra_body={"agent_reference": {"name": AGENT,
                "type": "agent_reference"}})
for _ in range(6):
    calls = [o for o in response.output
             if o.type == "function_call"]
    if not calls: break
    outputs = [FunctionCallOutput(
        type="function_call_output", call_id=c.call_id,
        output=data_tools.execute(c.name, c.arguments))
        for c in calls]
    response = openai.responses.create(
        conversation=conv_id, input=outputs,
        extra_body={...})
```

Tools: get_invoice_processing_stats · get_recent_invoices · get_supplier_price_deviations · get_sales_summary · get_margin_trend

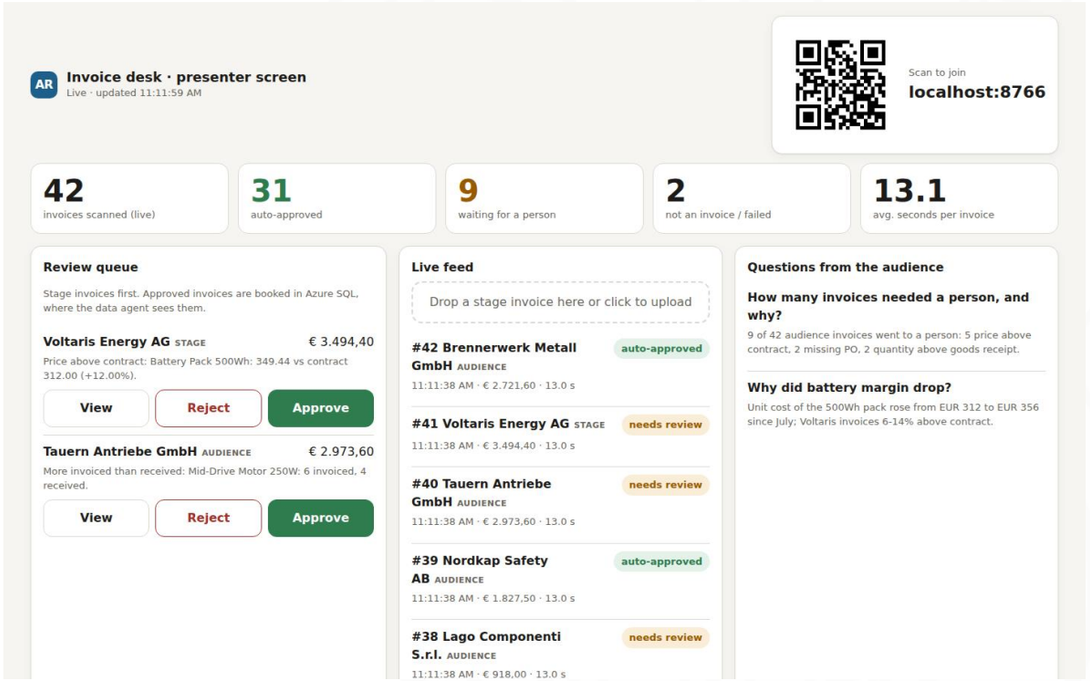
The front end



Phone: result



Phone: ask the data



Presenter screen: counters, review queue, live feed, questions

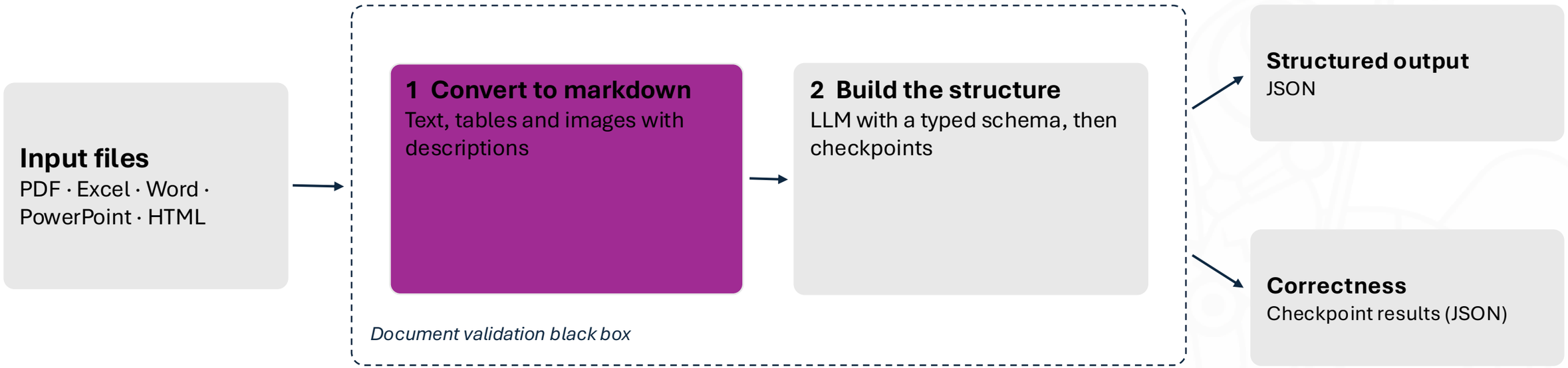
Endpoint	Used by
POST /api/invoices	Phone and presenter upload
GET /api/invoices/{id}	Phone polls every 1.5 s
POST /api/ask	Ask the data
GET /api/feed · approve · reject	Presenter (key), every 2 s

Demo: Real world issues

From documents to structured data

Real-world issue 1: getting reliable data out of PDFs, scans and Office files

Two steps: convert, then structure



- Most formats: Python libraries are enough
- **PDF is the hard part**: text inside images, scanned pages, complex tables
- Output of step 1: markdown plus a list of tables and images with descriptions

Validating what was extracted

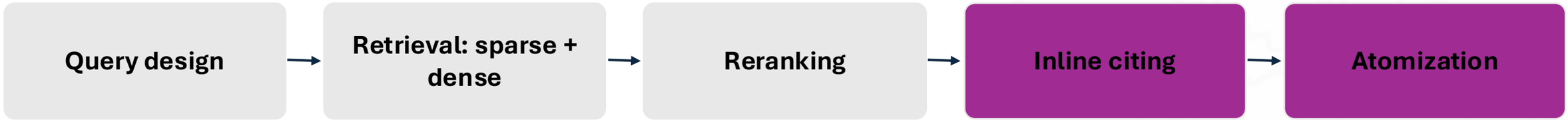
Real-world issue 2: can we trust the numbers?

Why trust is hard, and where to improve

Hallucinations
Plausible, but not in the source documents

Non-determinism
Same input, different output

Black box
Unclear why the model answered this way



Points of improvement, from question to answer

- Sparse retrieval catches exact words and numbers, but not synonyms: **never use it alone**
- Atomization: check each claim ('$20', scope 'entrance fee') instead of whole sentences
- **Correct numbers matter most**

Notes from the field: caveats and problems

Notes from the field

- Variety of models, functionality and language support is great
- Model availability differs across Azure regions
- Deployment issues
- The Foundry interface can sometimes be puzzling
- SDK and IaC (Bicep) are great, but feel incomplete
- A lot of features are still in preview
- Is Foundry a platform, a service or just a GUI?
- Lift and shift, and exit scenarios, with Foundry
- Prompty language

Caveats people run into

- Two sets of documentation on Microsoft Learn
- SDK and API versions change (1.x vs 2.x); namespaces and functions differ
- Hub-based vs Foundry projects, and multiple resource types
- Model deployments are separate, even for the same model
- Standard deployments: regional limits on models and throughput; latency varies at high volume
- Models have a lifecycle: plan for retirements
- Generated content: hallucinations, non-deterministic output, black box
- Validation: rules are labour-intensive; LLM-as-judge is as untrustworthy as the LLM it checks

Key dates and retirements



Date	What	Move to
20 Aug 2026	Initial-preview hosted agent backend	Redeploy on the current hosted agents
26 Aug 2026	azure-ai-inference package	openai package, v1 endpoint
26 Aug 2026	Assistants API	Responses API and Foundry agents
1 Dec 2026	Declarative workflows in Foundry	Microsoft Agent Framework
20 Apr 2027	Prompt flow (portal, VS Code, runtime images)	Microsoft Agent Framework
31 Mar 2029	Language: sentiment, key phrases, summarization, entity linking, CQA, CLU	See the Language migration guidance

Get started



- Create a Foundry resource and project in ai.azure.com
- Install Foundry DevPack (azd, Toolkit, Skill, Canvas)
- Build a prompt agent in the portal, then the same in code
- Add a Foundry IQ knowledge base and a toolbox
- Ship a hosted agent with azd ai agent deploy
- Turn on tracing and evaluations from day one

- learn.microsoft.com/azure/foundry
- learn.microsoft.com/agent-framework
- github.com/microsoft-foundry/foundry-samples
- github.com/microsoft/foundry-dev-tools
- devblogs.microsoft.com/foundry



Thank you!

Let's connect

http://tomaztsql.wordpress.com

tomaz.kastrun@gmail.com

@tomaz_tsql

/in/tomaztsql

http://github.com/tomaztk

https://mvp.microsoft.com/PublicProfile/5002196

Session material:

github.com/tomaztk/Azure-AI-services